Section 1 – Getting Started

Description: Access PowerApps and get started with creating your component.

Overview: Open the PowerApps maker portal. Navigate to the environment where you will be building the component library. Create a new component library.

Tips & Tricks

(1) Click the "…" in the left navigation bar to see Component Libraries if you don't have it pinned. It should be in the Discover All panel.

Section 2 – Create the Modal Window

Description: Create and configure the outermost containers and a rectangle that will be used in your component. We will be using (2) container controls to help build this component. We will populate the Fill properties for each container to create the modal window.

Overview: Create your 2 containers and configure the properties accordingly. Fill properties can be configured as desired. The below is are the recommended configurations.

Cntr_OuterModal

Fill: RGBA(140,140,140,50%)

Height: Parent.Height

Width: Parent.Width

Cntr_InnerModal

Fill: RGBA(255,255,255,1)

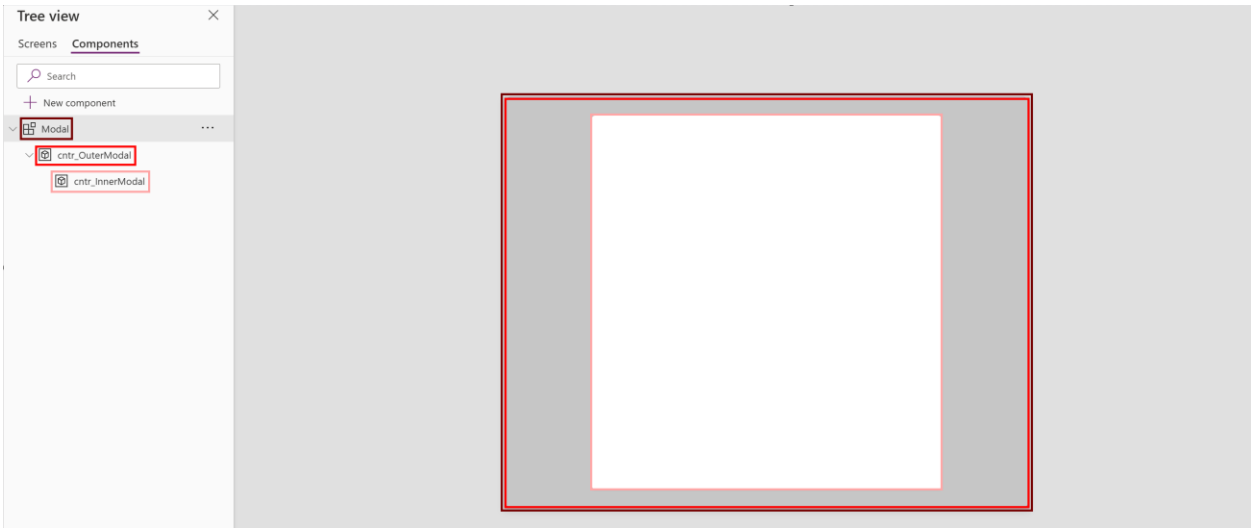
Height: Parent.Height / 1.1

Width: Parent.Width / 1.5

X: (Parent.Width - Self.Width) / 2

Y: Self.Height / 20

You should end up with your component looking something like the following:



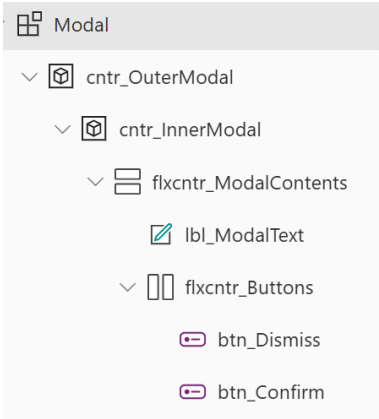
Tips & Tricks

(1) Getting your outermost and innermost containers sized (height/width) and positioned (x/y) is critical to having a scalable modal window that works on all apps. We will use the Parent method to achieve this.

Section 3 – Add Modal Window Content

Description: We are now ready to start adding some content to our modal window. This will require some flexible vertical and horizontal containers, some buttons and a text label.

Overview: Create (1) flexible vertical container. Create 1 text label. Create 1 flexible horizontal container. Create 2 buttons. These controls will be nested, and should look like the following:



Flxcntr_ModalContents

DropShadow: DropShadow.None

Height: Parent.Height

LayoutAlignItems: LayoutAlignItems.Start

LayoutJustifyContent: LayoutJustifyContent.End

Padding (Top,Bottom,Left,Right): 10

Width: Parent.Width

Lbl_ModalText

Align: Align.Center

AlignInContainer: AlignInContainer.Start

AutoHeight: true

FillPortions: 1

VerticalAlign: VerticalAlign.Bottom

Width: Parent.Width - 20

Flxcntr_Buttons

DropShadow: DropShadow.None

Height: 50

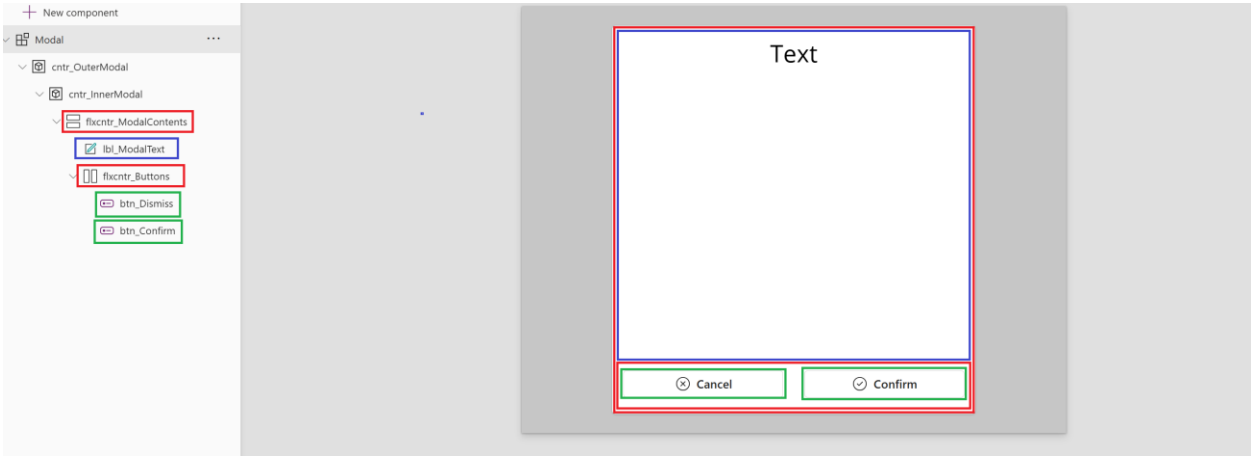
LayoutGap: 25

LayoutAlignItems: LayoutAlignItems.End

Padding (Left, Right): 3

Width: Parent.Width

Try walking through and configuring the 2 remaining button controls. Once complete, your component should look something like the following:



Section 4 – Enhancing the Component Library

Description: Enable 'Enhanced component properties' via Settings → Experimental tab in PowerApps. This is required to handle the events that occur when our buttons are clicked. We will then create our first component property to control the visibility of our modal window.

Roadmap

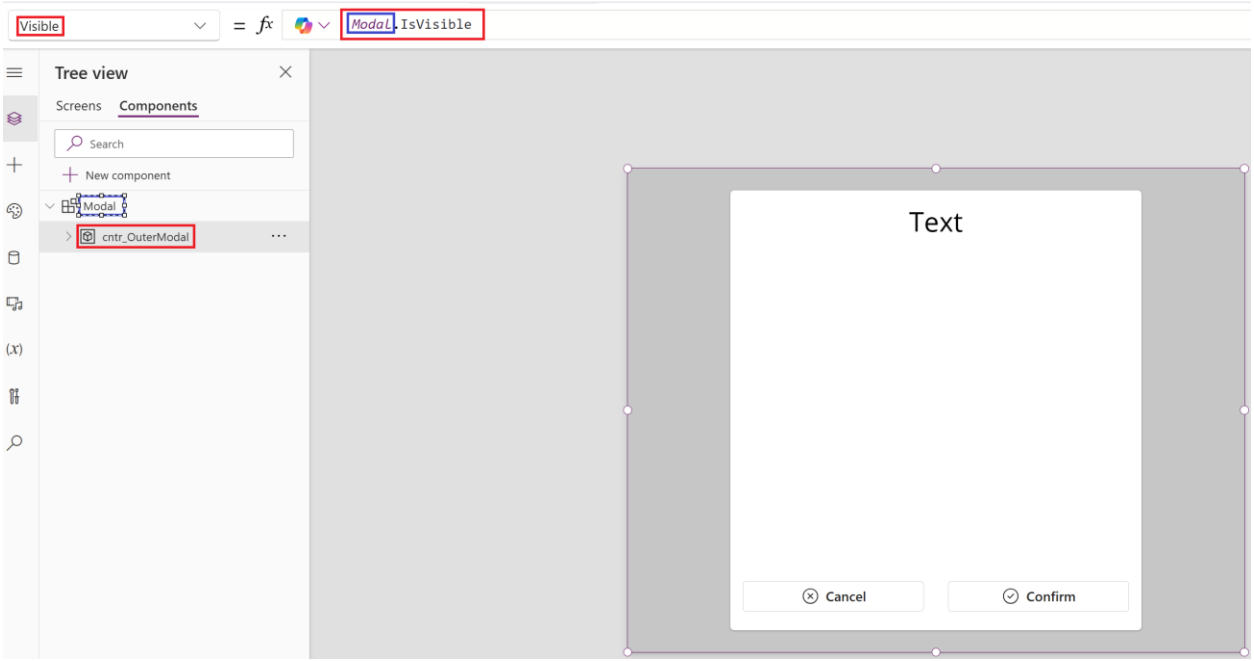
Enhanced component libraries are an experimental feature. Not all component libraries require this feature. For this workshop, we enable this feature to show the capabilities on all property types, including those in Preview. Ensure that your applications do not have a critical dependency on a component library that is using experimental features until that feature has become Generally Available.

Enhanced Component Properties release timeline (powerplatform.com)

Overview: Create a new custom property on your component. Configure your outermost modal container Visible property to the recently created component property. Add a new screen to test your component. Validate the show and hide capability based on an event (i.e. button click).

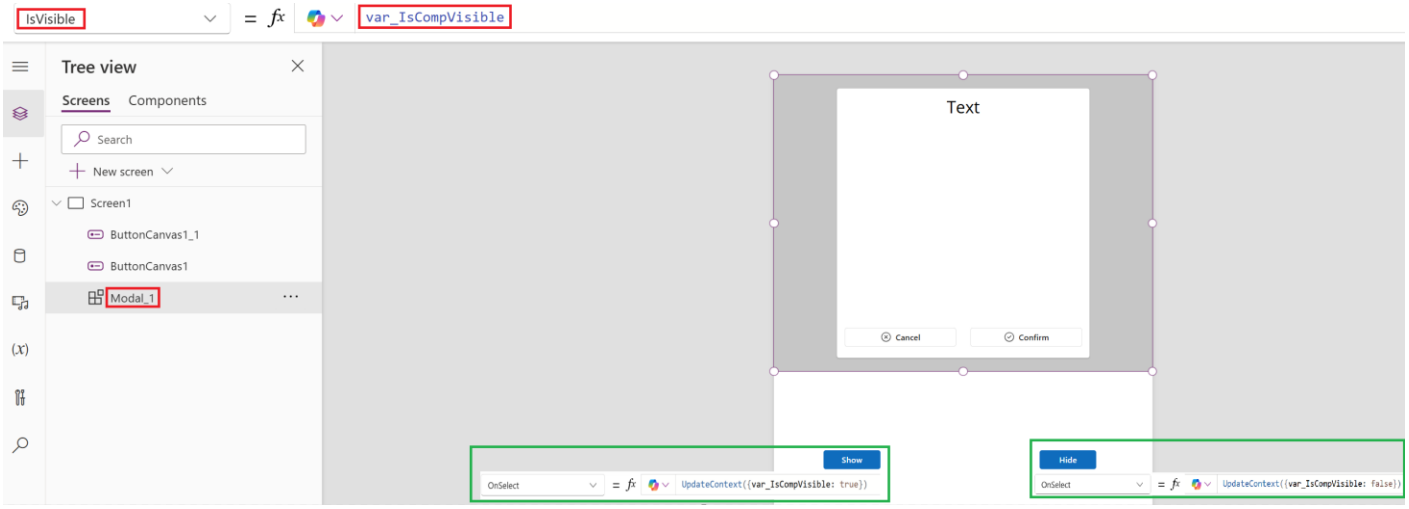
Details: Create your component property. Set the Name to 'IsVisible' or something similar to indicate what the property is for. Add a description and set the Property type to 'Data' and definition as 'Input'. Finally, set the Data type to Boolean.

Configure the modal to show/hide depending on the value of our recently created component property. To do this, we set our cntr_OuterModal 'Visible' property to <ComponentName>.<ComponentProperty>



Try toggling the component property you created from true to false and vice versa. You should see your component show and hide accordingly.

We now need to actually test our component behavior in a typical app experience. Create a new screen, configure the IsVisible property to a variable that we will set via 2 buttons.



Show Button; OnSelect: UpdateContext({var_IsCompVisible: true})

Hide Button: OnSelect: UpdateContext({var_IsCompVisible: false})

Tips & Tricks

(1) Remember, only objects in the 'Components' tab in your component library will exist in your component library. Screens are simply for testing. This allows you to test your component directly in your component library app without needing to continually publish to another app.
(2) The standard Visible property is not accessible in the component view but does exist when adding the component to your app. For this reason, we created our own 'IsVisible' property so that we can validate behavior directly in our component. You can still use the standard Visible property from within your app where the component is hosted.

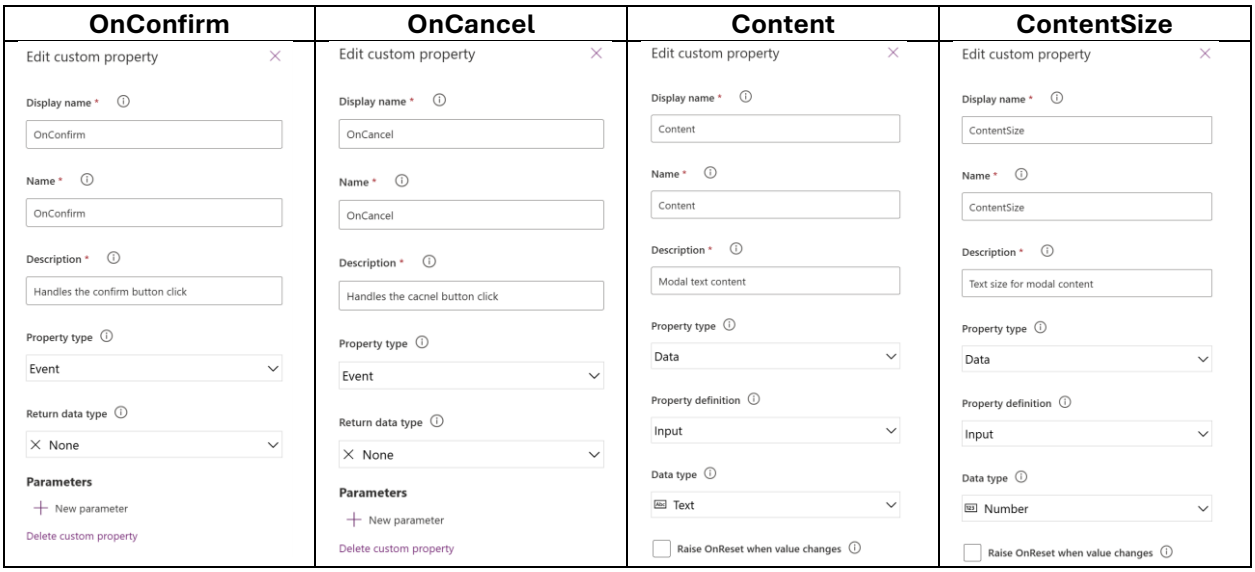
We have demonstrated that ability to create our first component property and make it useful in an application using a variable that we can control from the app hosting the component library. Next, we will look to further enhance the component by configuring our cancel and confirm buttons. Finally, we will allow for dynamic modal text to be passed in from the app hosting the component.

Section 4 – Finalizing the Component Library

Description: We are now ready to finalize our component library by adding a few more properties.

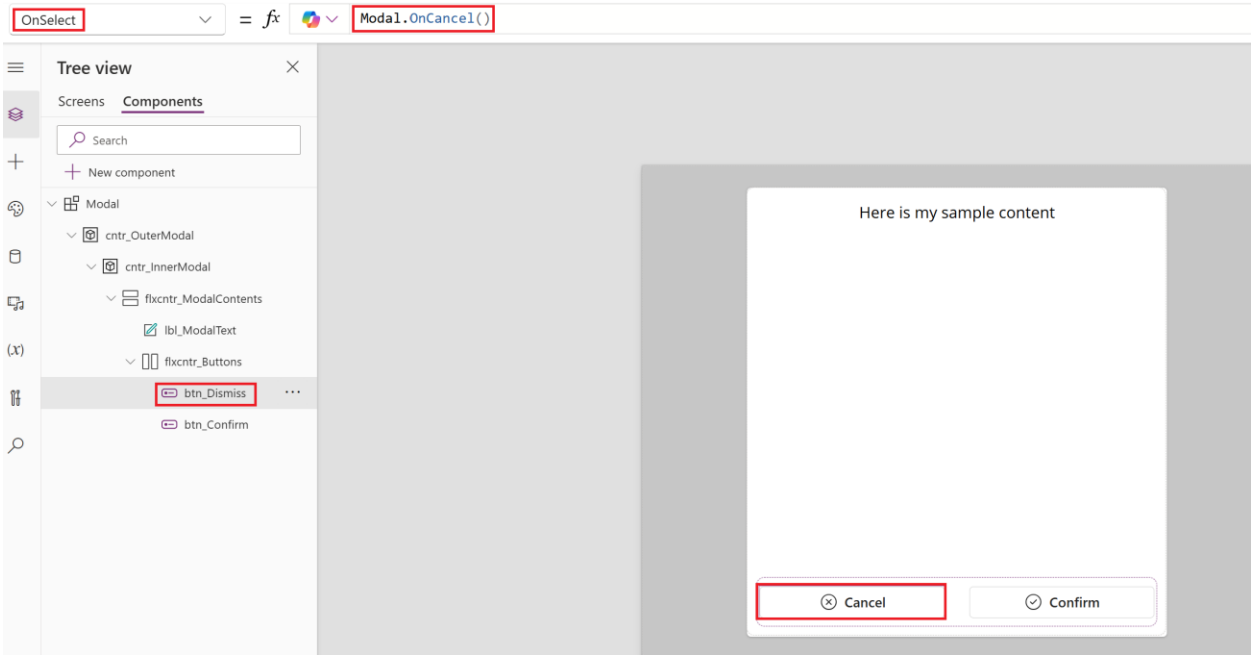
Overview: Create our 4 more component properties to handle the behavior of our cancel and confirm buttons as well as dynamic text / sizing for the modal content. Bind the new properties to our buttons in the modal and text label for modal content. Test button behavior and dynamic text / sizing in app.

Details: Using similar steps that we used to create the IsVisible property, create 4 more new component properties. See below for details and configuration



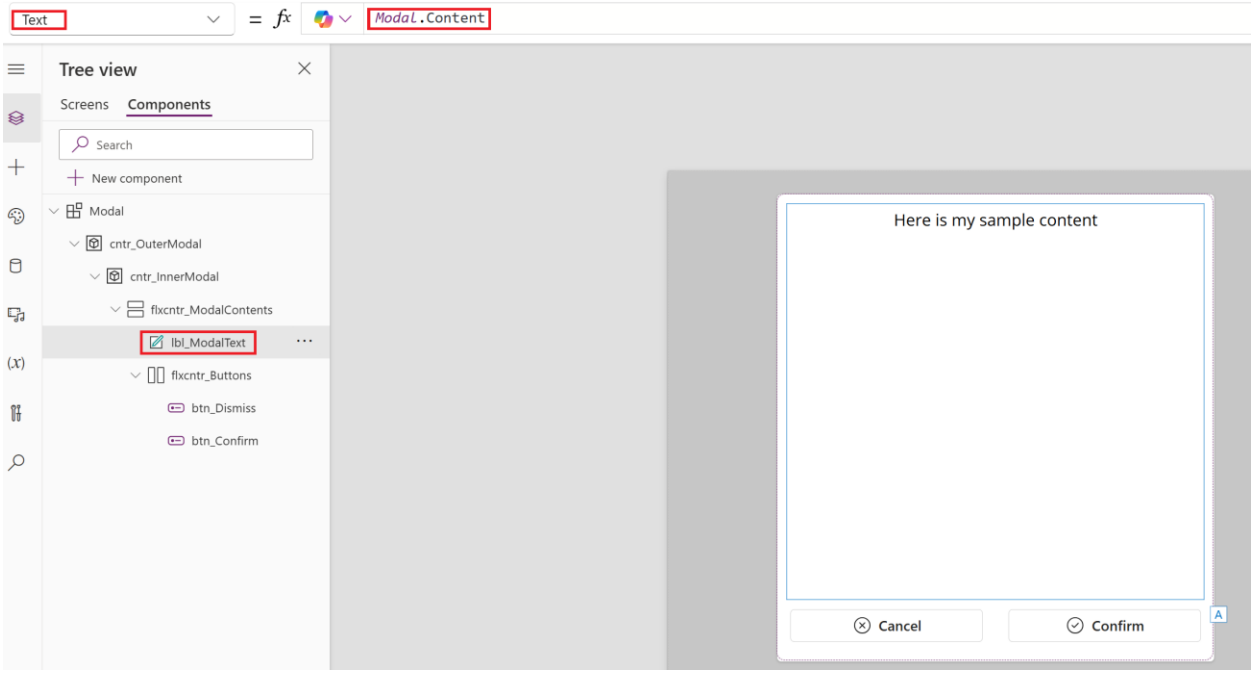
OnConfirm and OnCancel will be Event property types. Content will be Text and ContentSize will be number.

Configure the OnSelect of our Cancel button to hook into the OnCancel event property from our component.



Repeat these steps for the Confirm button and point to our OnConfirm property.

Configure our modal Text label to our Content property.



Repeat these steps for the Size property.

Go back to your Screen where we will now configure the behavior of our new properties. Take the code from the Hide button and paste within the OnCancel property. This will now hide the component. Configure the behavior when the confirm button is click. For demo purposes, we will call a Notify() function.

Repeat these configuration steps for the Content and Content Size properties. Notice that the modal content text and sizing adjust accordingly.

Tips & Tricks

(1) You can use the OnConfirm button to handle several different actions. This includes accessing app data for things like deleting an item or navigating to a different screen.
(2) You can add the same component *multiple* times in the same app, just like other controls. Don't forget to utilize different variables for each component should you add more than one to your app.

Section 5 – Publish and Share

Description: Our component library is complete. We are now ready to make 1 more publish and share the component library (if applicable).

Overview: Publish your component library. Share the component library if needed. Test adding your component library to an existing or new canvas app. Demonstrate the ability to update your component library.

Tips & Tricks

(1) Component libraries are environment specific, just like apps and flows. Take note of where your component library lives. Only canvas apps in that environment can use your component library.

Section 6 – Add and Configure the Component Library

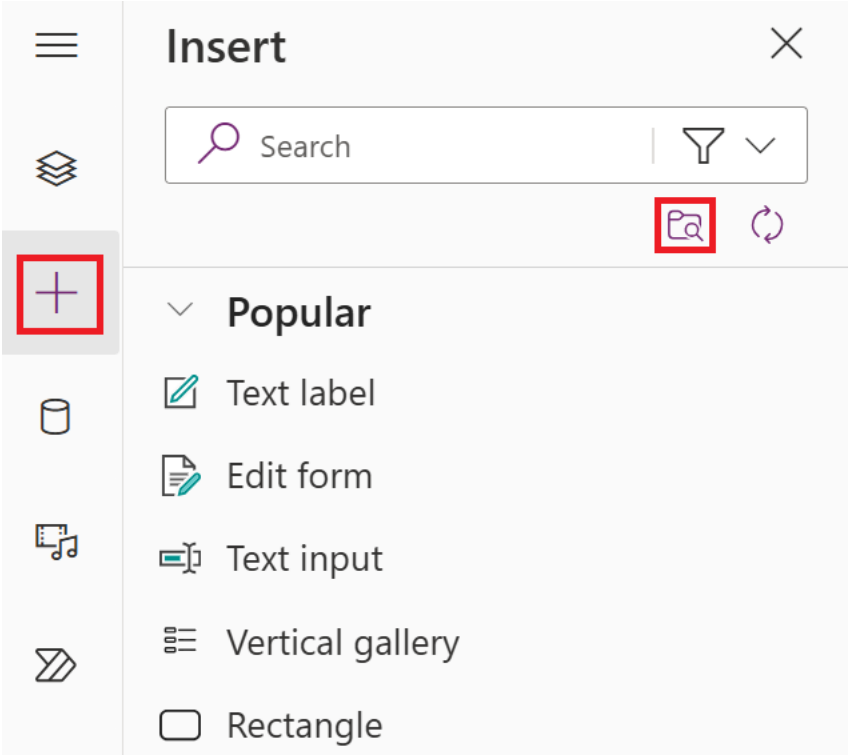
Description: Using an existing or new canvas PowerApp, we will add our published component library within the app.

Sample Demonstration

(1) The steps in this section will be app specific and will vary based on the application where you are using the modal. This guide will utilize the component to confirm deletion of a Dataverse record. It is recommended that for this workshop, you use the modal for a similar function.

Overview: Create or use an existing PowerApp. Ensure this app is in the same environment as your library. Import the component library. Configure the component library. Test the component library in your app.

Details: In your canvas app, import your component

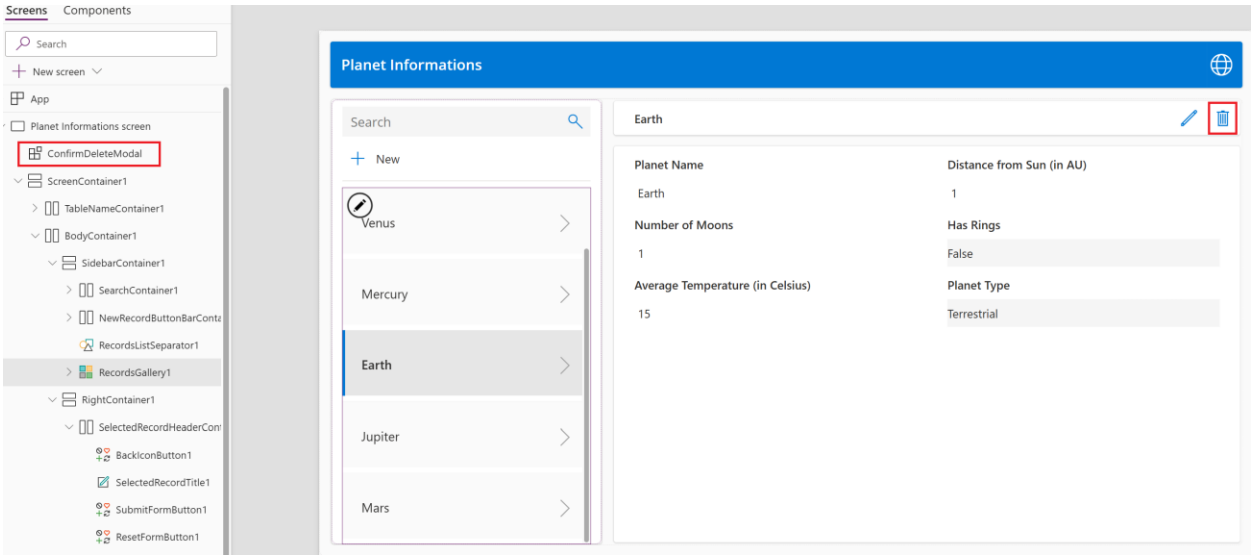


Select the component and click 'Import'. Add the component to a screen. You should see the component under 'Library Components'.

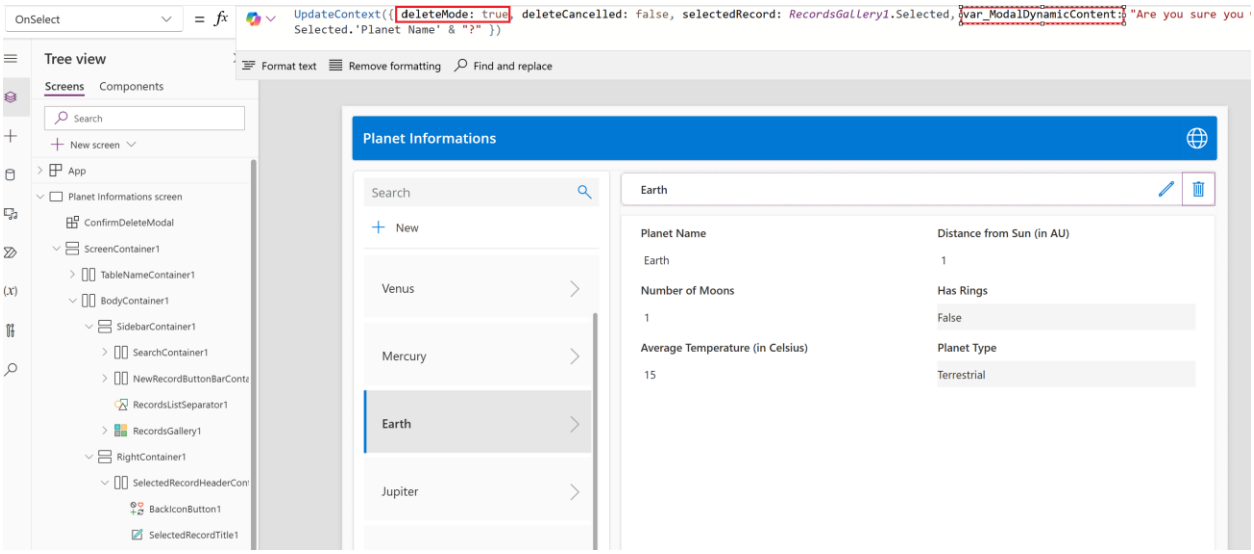
Tips & Tricks

(1) Height and Width not right? Try and update those properties to inherit from the parent.

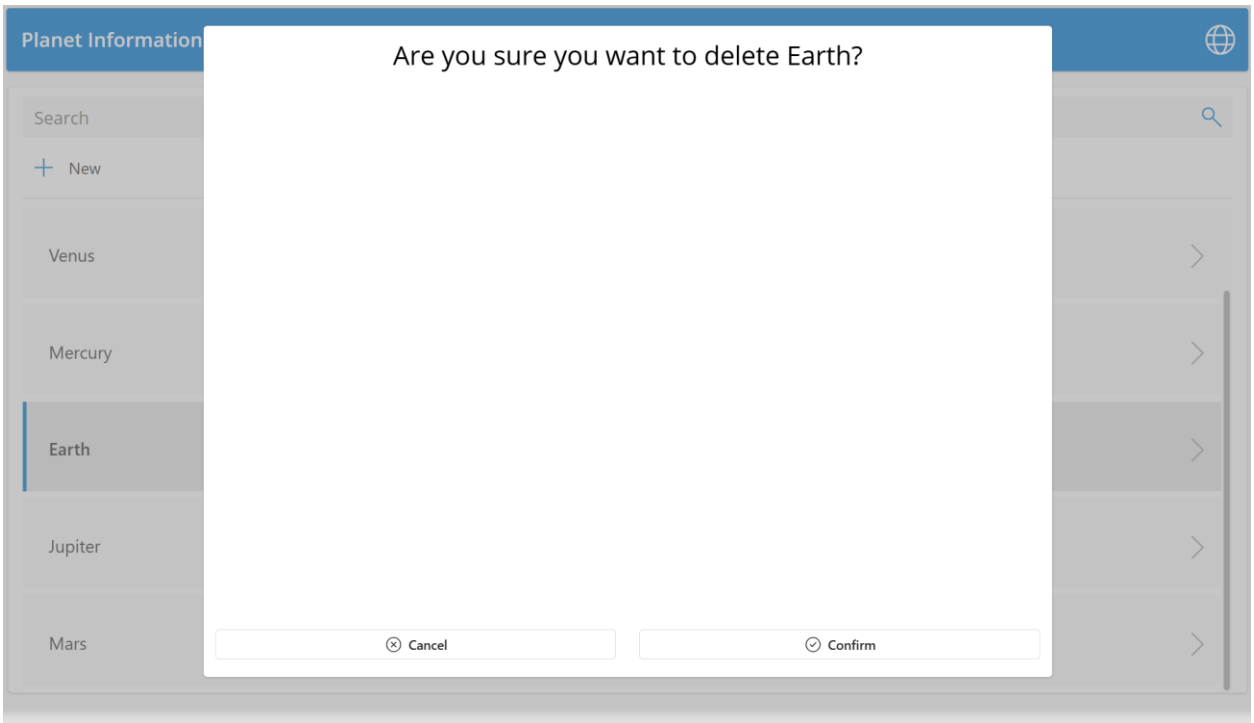
Configure your component properties to behave as desired within your application.



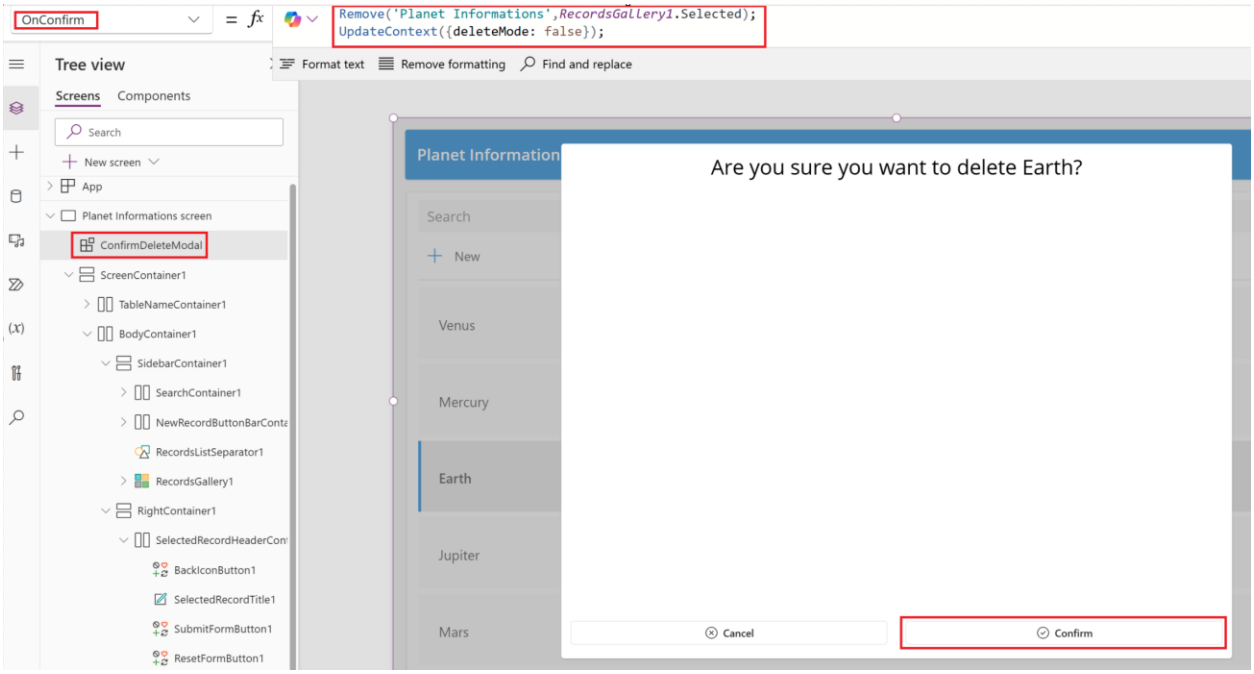
In the above example, I have hooked into the pre-existing delete button and set ‘deleteMode’ and var_ModalDynamicContent accordingly. See below



Upon clicking the delete button, my modal now shows because deleteMode = true. I also see my dynamic modal content.



I have configured the Cancel button to set deleteMode back to false. I have chained some actions for the Confirm button. See below



When Confirm is clicked, we remove a dataverse record and set deleteMode back to false, hiding our component.

Tips & Tricks

(1) In your component, ensure the Visible and IsVisible properties are set to the same value. Once you are comfortable working with your component, you can shift to only using the Visible property.

Section 6 – Update the Component Library

Description: Demonstrate the ability to show an updated version of the component library in your canvas app.

Overview: Think of a capability you can add or change in your component and update your component. Republish the component. Pull the updated version of the component into your canvas app. Republish your canvas app. Validate the new component feature(s) in your published canvas app.

Stumped for ideas? Check the bonus section out below.

Bonus (optional)

(1) Scenario: Users are still accidentally deleting records even with the modal window you deployed. Application owners would really like a text box on the modal window where the user must type the name of the record they want to delete.

Section 7 – Bonus | Text Input Confirmation (Optional)

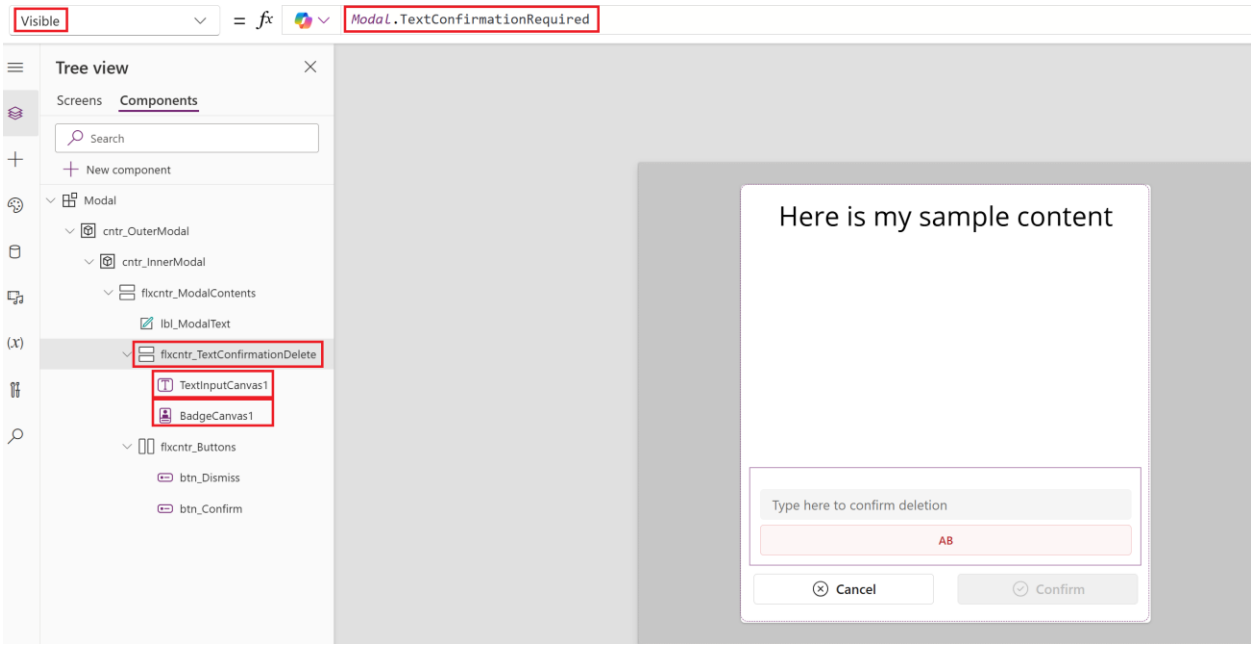
Description: Reduce accidental deletions by incorporating forcing users to type the name of the record before deletion. Make this an optional capability in your component.

Overview: Create new controls and properties as needed to add this capability.

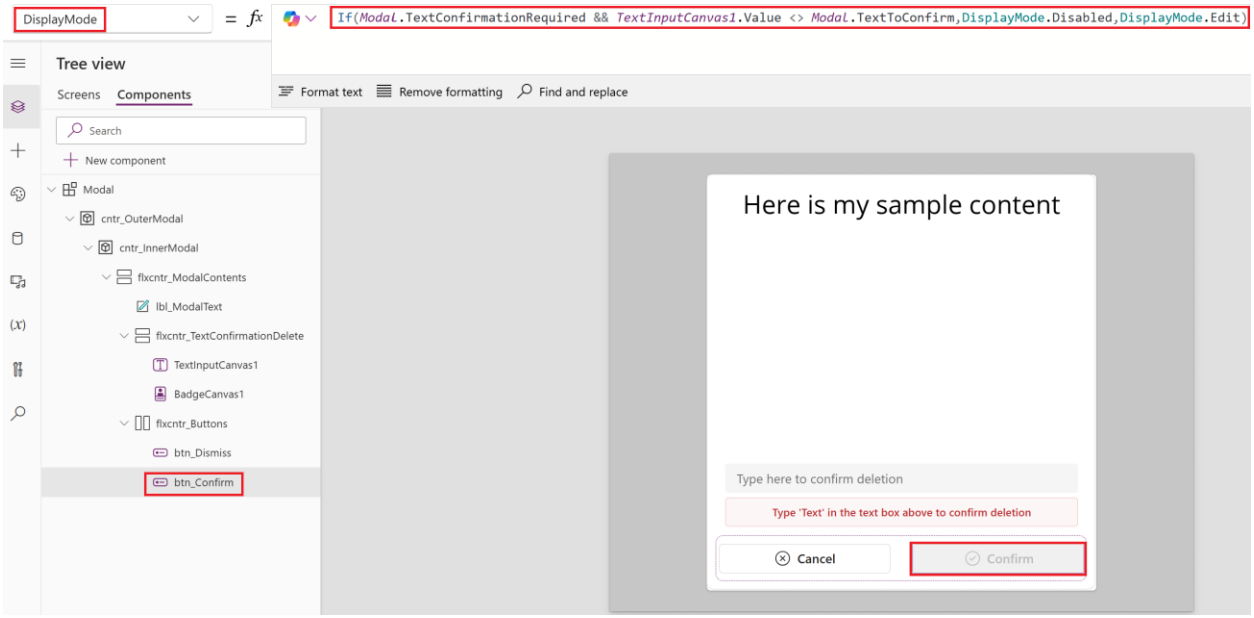
Details: Create 2 new properties called TextConfirmationRequired and TextToConfirm. We will use these to show/hide our new controls and determine the DisplayMode of your Confirm button.

TextConfirmationRequired	TextToConfirm
Edit custom property Display name * TextConfirmationRequired Name * TextConfirmationRequired Description * Used to determine if our controls for text confirm... Property type Input Data type Boolean Raise OnReset when value changes Delete custom property	Edit custom property Display name * TextToConfirm Name * TextToConfirm Description * Text to match to enable confirm button. Only app... Property type Input Data type Text Raise OnReset when value changes Delete custom property

We conditionally determine if flxcntr_TextConfirmationDelete should be visible based on the component property.

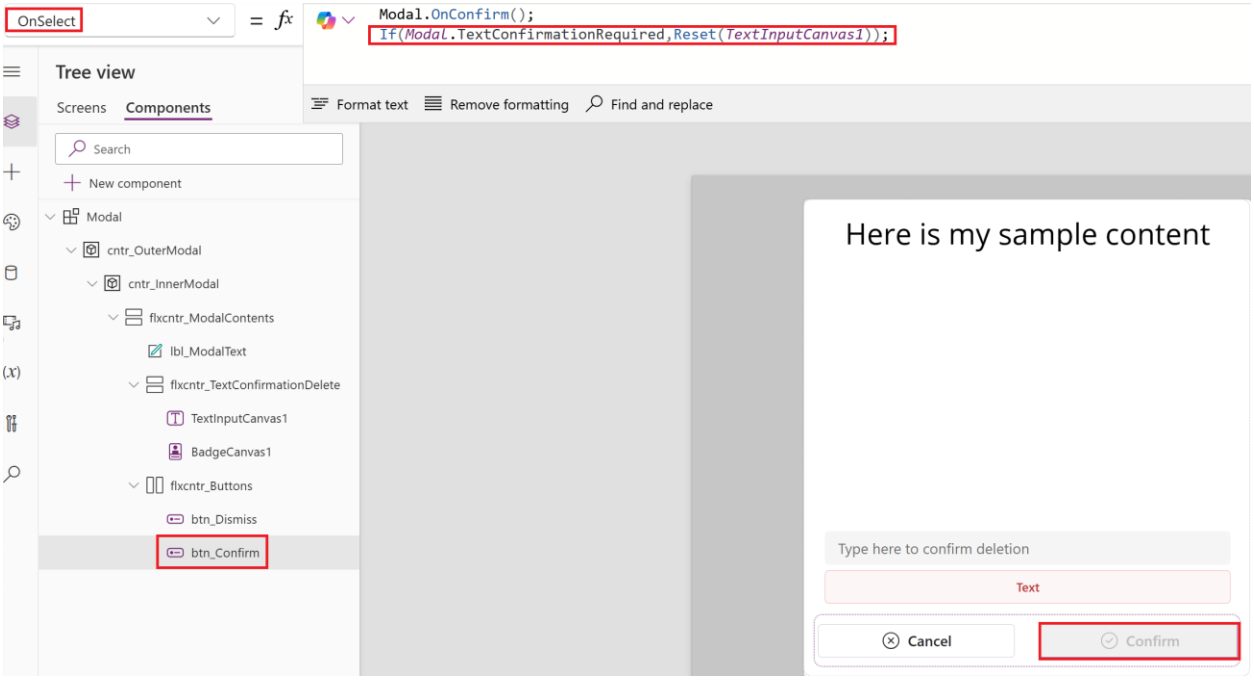


We conditionally determine whether the Confirm button should be enabled.

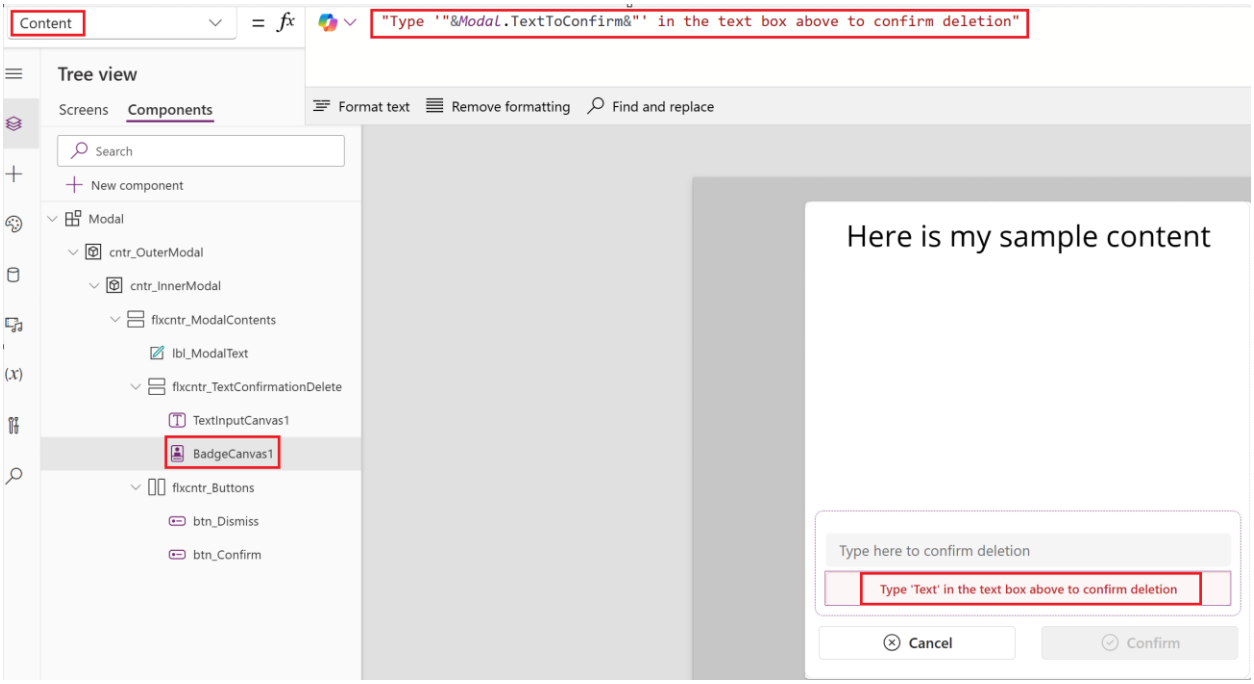


We set our badge content to our component property. This will be used in app to show the text the user must type in order for the Confirm box to re-enable.

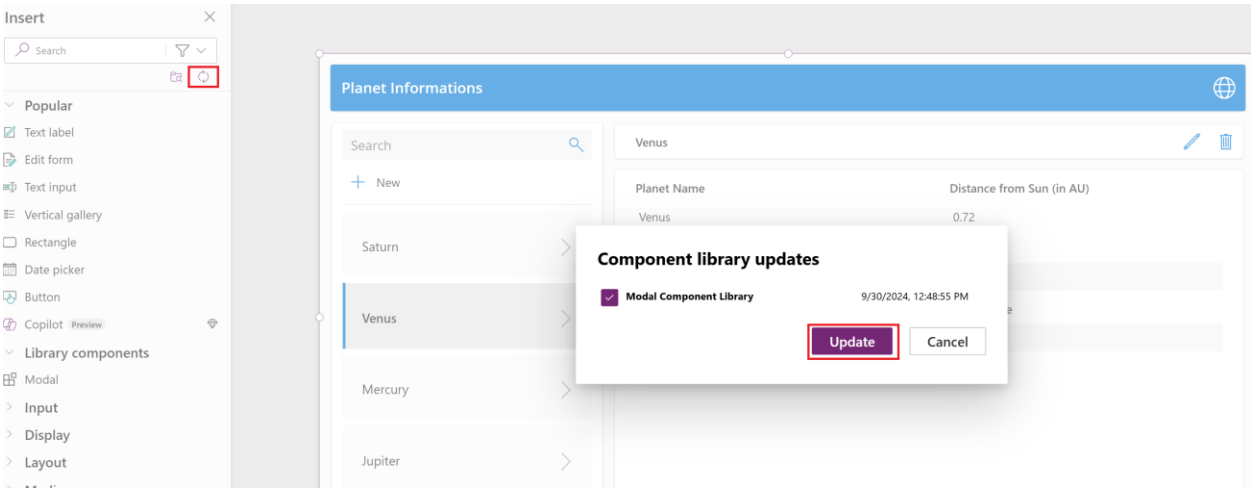
Update the OnSelect of our confirm button to reset the text input. This ensures that the next time the modal shows, existing text will not be displayed.



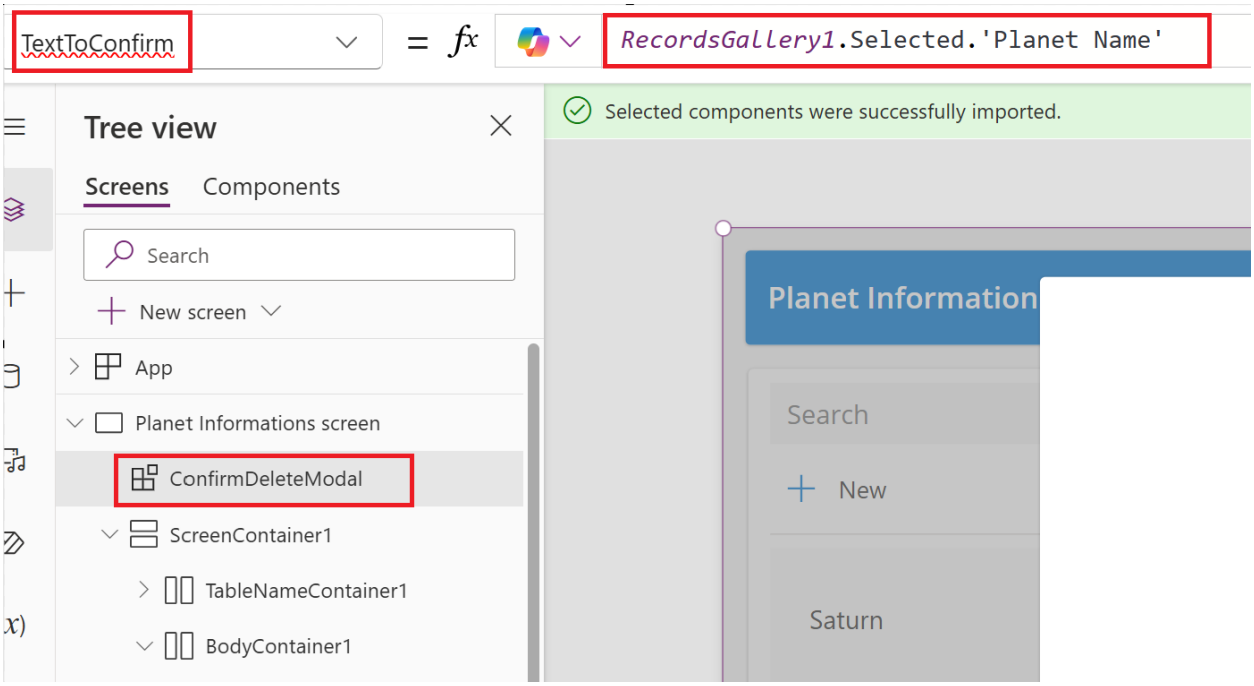
Finally, update the Content of our badge control to give the user details on what to type.



Republish the component. Pull the updated version of the component into your canvas app. Republish your canvas app. Validate the new component feature(s) in your published canvas app.

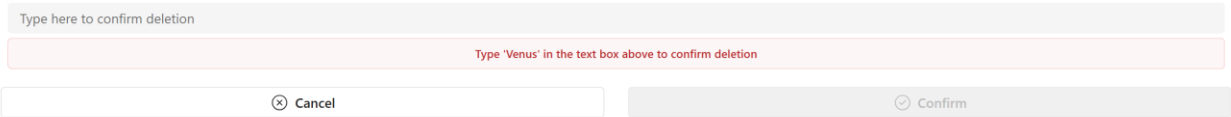


Configure the new properties in your canvas app. In this demo app, we configure the 'TextToConfirm' property to the name of the selected record.



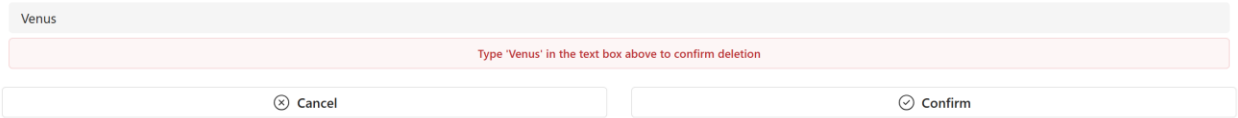
Notice that the Confirm button is not enabled until we type the name of the record to delete in the text input box.

Are you sure you want to delete Venus?



After we type the record name, the confirm box is enabled.

Are you sure you want to delete Venus?



Tips & Tricks

(1) Avoid publishing your component until you are ready for makers to update in their canvas app. Once you publish a new version, makers will be notified via a banner in their app hosting your component that an update is available.

(2) You do not need to re-add the component back on your screen. All existing components on all screens are automatically updated.